

CSE480: Web Security

5. Same Origin Policy & Cookie

Seongil Wi

Recap: Web Threat Models

- **Network attacker:** resides somewhere in the communication link between client and server
 - Passive: eavesdropping
 - Active: modification of messages, replay...
- **Remote attacker:** can connect to remote system via the network
 - Mostly targets the server
- **Web attacker:** controls attacker.com
 - Can obtain SSL/TLS certificates for attacker.com
 - Users can visit attacker.com



Today's Topic



- **Network attacker:** resides somewhere in the communication link between client and server
 - Passive: eavesdropping
 - Active: modification of messages, replay...
- **Remote attacker:** can connect to remote system via the network
 - Mostly targets the server
- **Web attacker:** controls attacker.com
 - Can obtain SSL/TLS certificates for attacker.com
 - Users can visit attacker.com

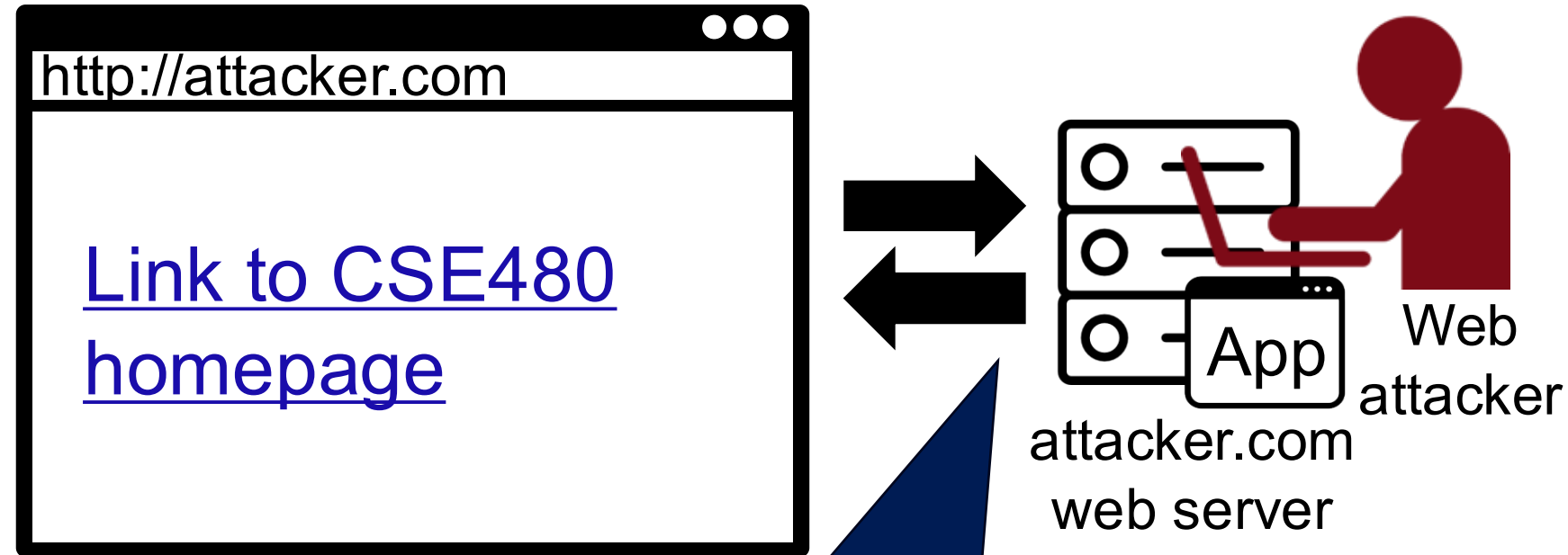




**Victim = End users
(Clients)**

Web Attacker

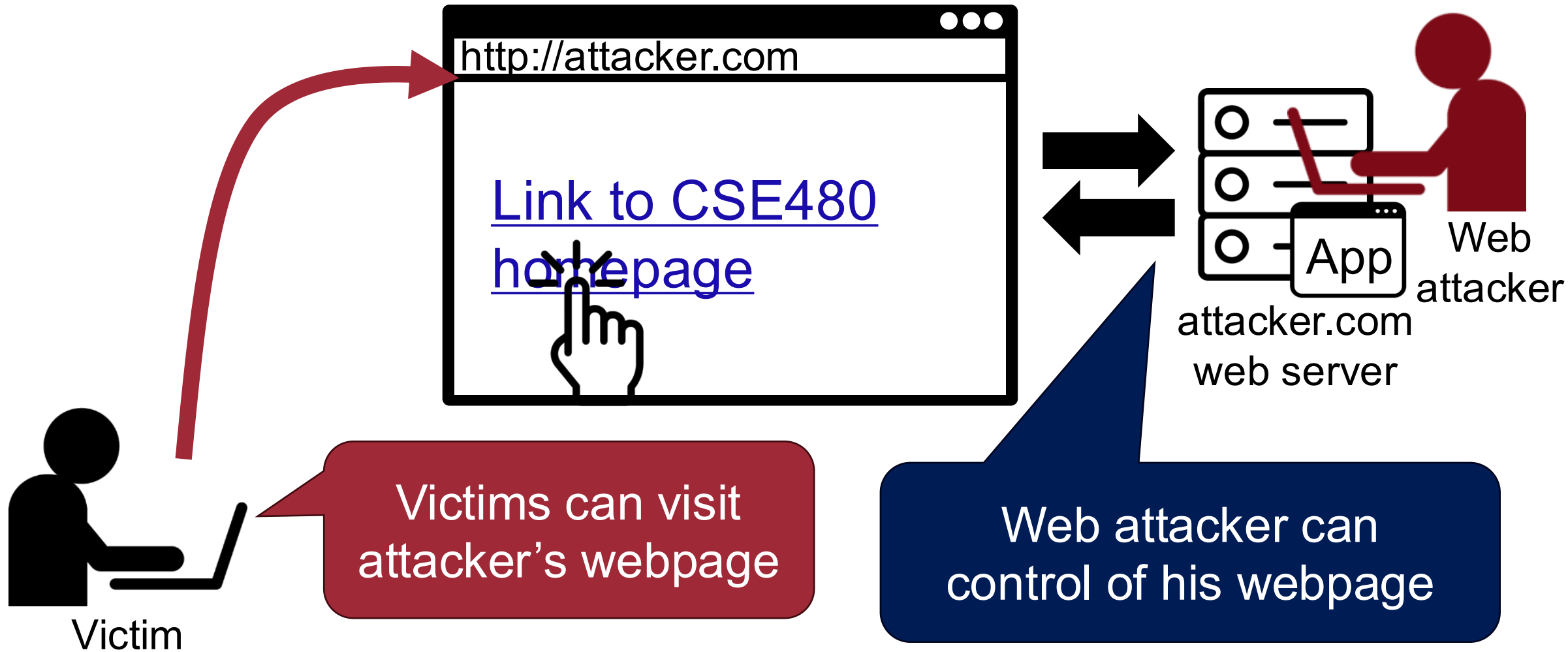
5



Web attacker can control of his webpage

Web Attacker

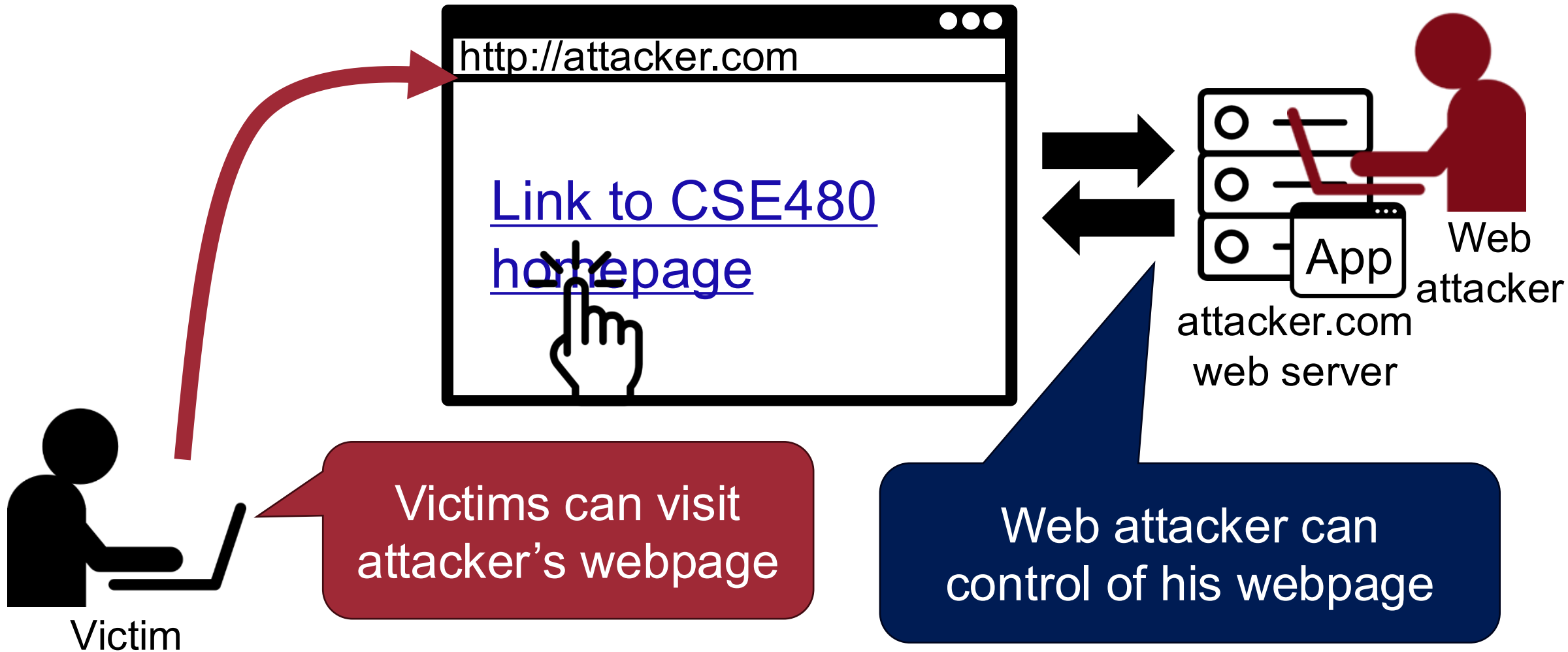
6





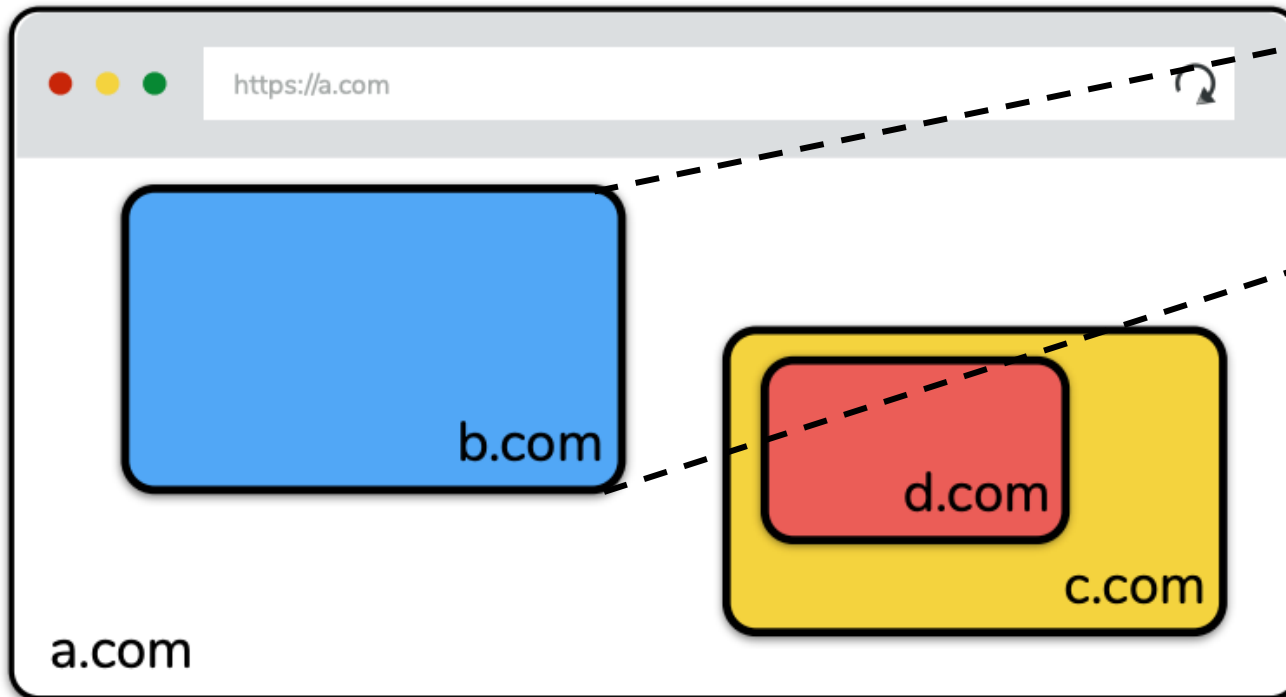
How it can be Possible?

7



Background: Nested Execution Model

- Windows may contain frames from different sources
 - **Frame**: rigid visible division
 - **iFrame**: floating inline frame



```
<iframe src="b.com">  
</iframe>
```

Background: Cookie

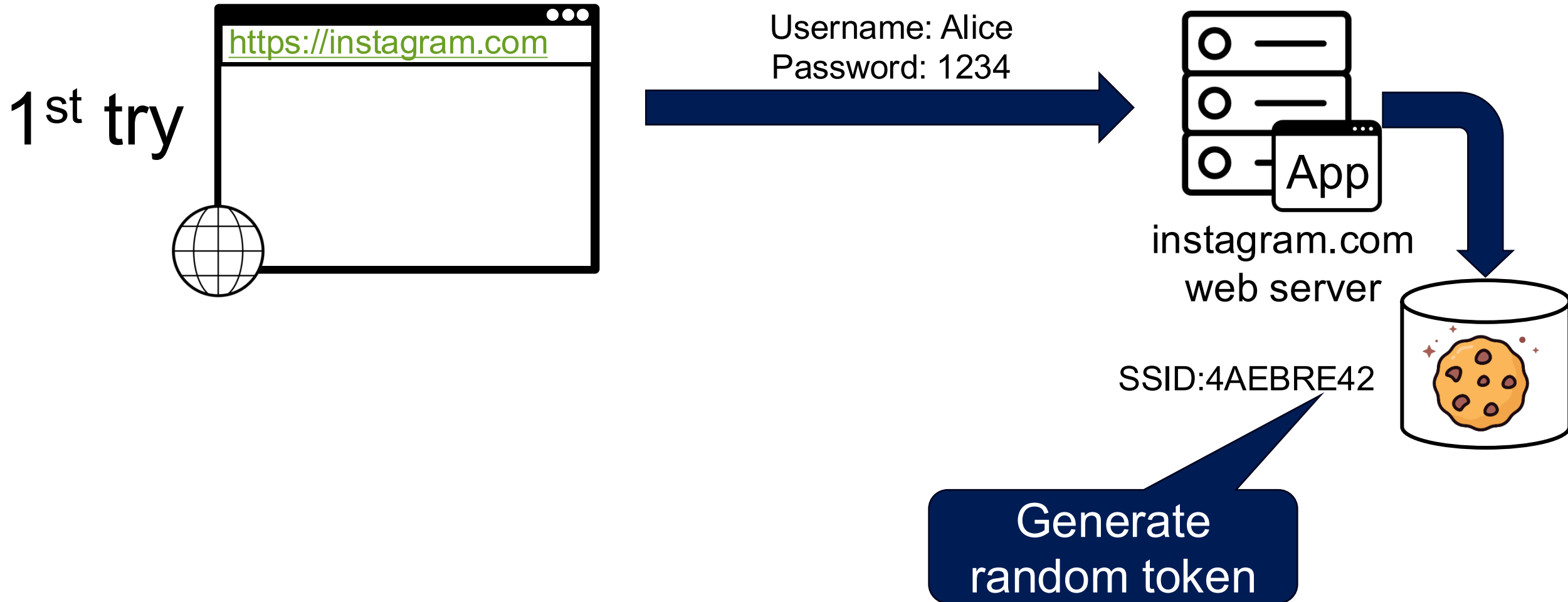


- HTTP cookie: small piece of data that a server sends to the browser, who stores it and sends it back with subsequent requests

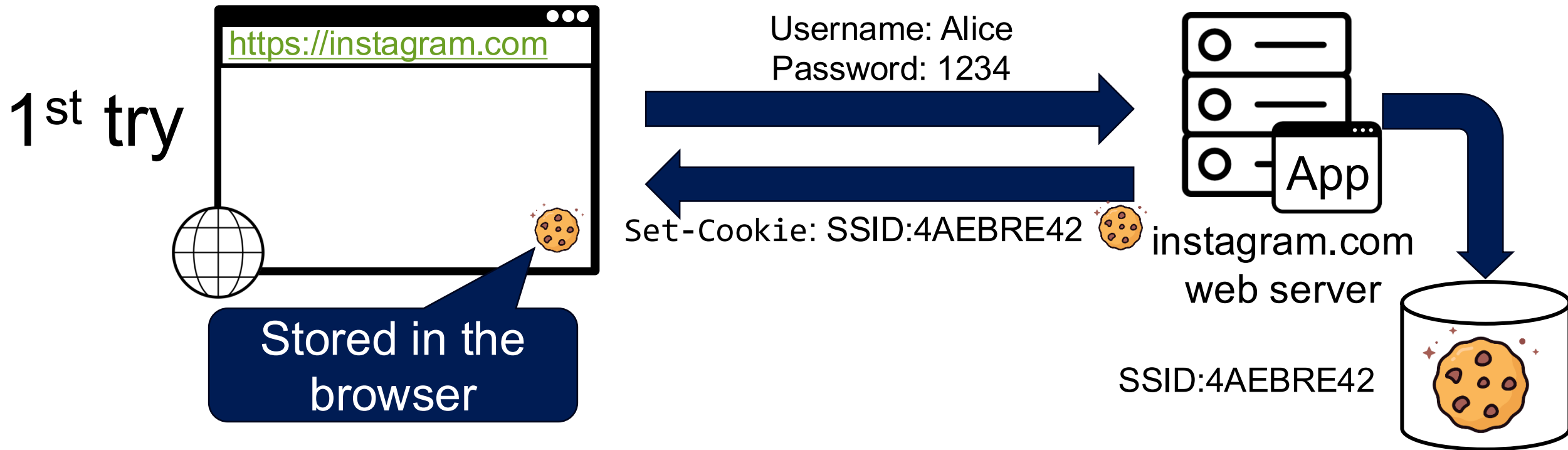


Cookie: Making HTTP Stateful

10



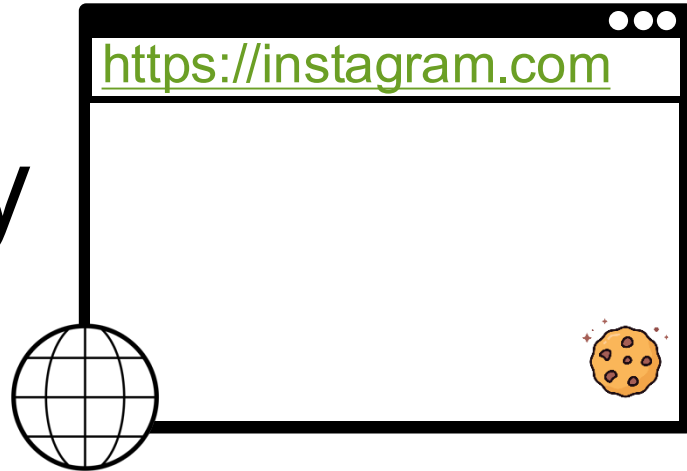
Cookie: Making HTTP Stateful



Cookie: Making HTTP Stateful

12

1st try



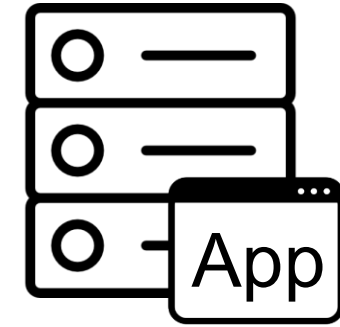
Username: Alice
Password: 1234



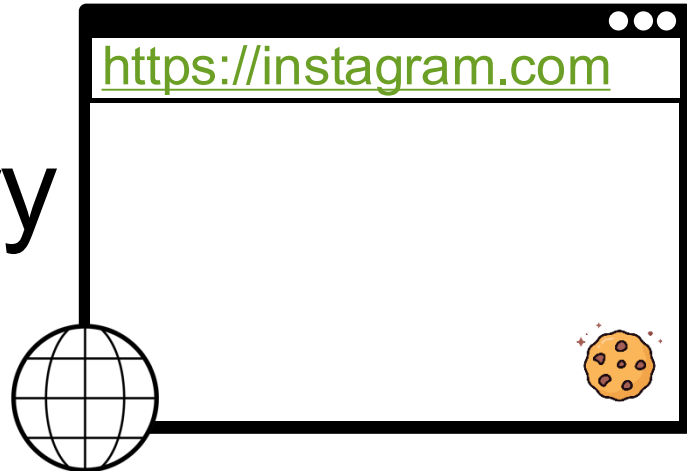
Set-Cookie: SSID:4AEBRE42



instagram.com
web server



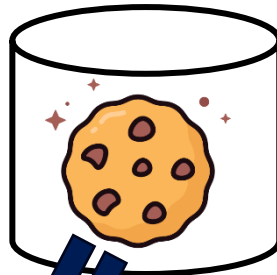
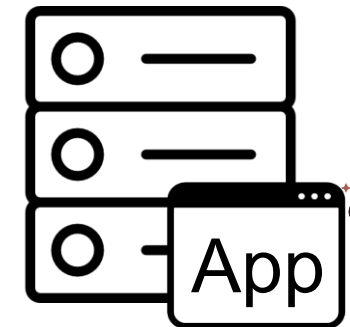
2nd try



Hey Instagram,
show me my profile
Cookie: SSID:4AEBRE42



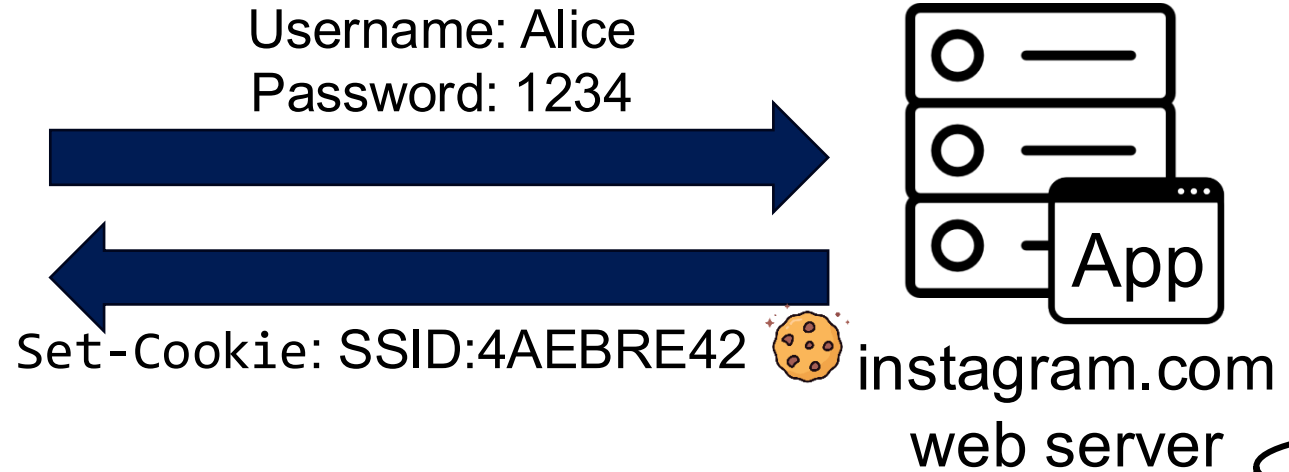
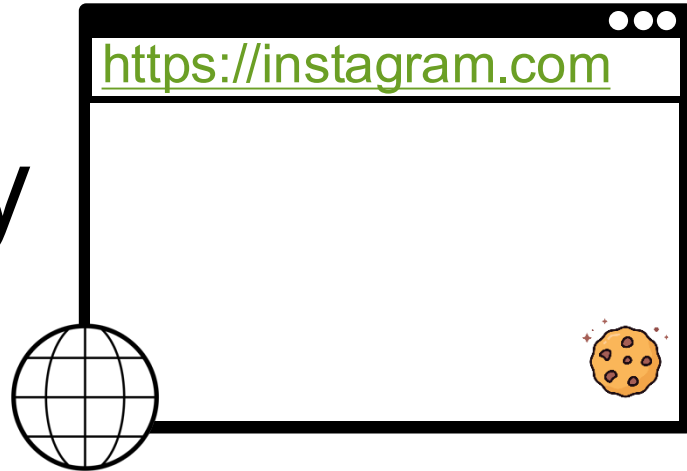
instagram.com
web server



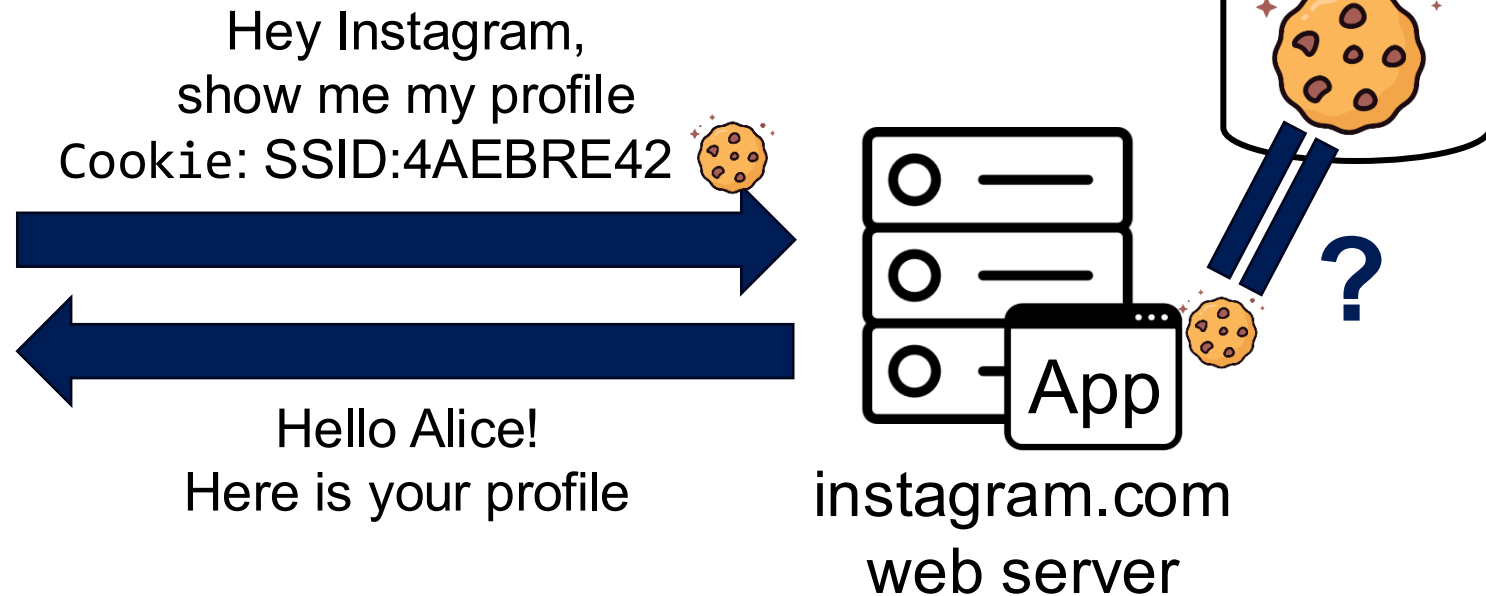
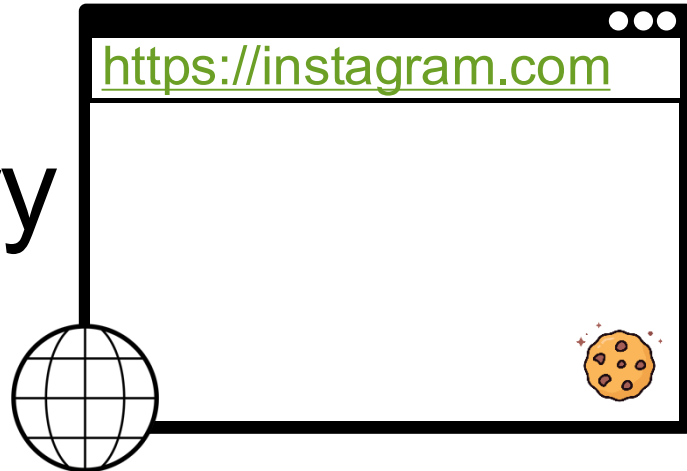
Cookie: Making HTTP Stateful

13

1st try

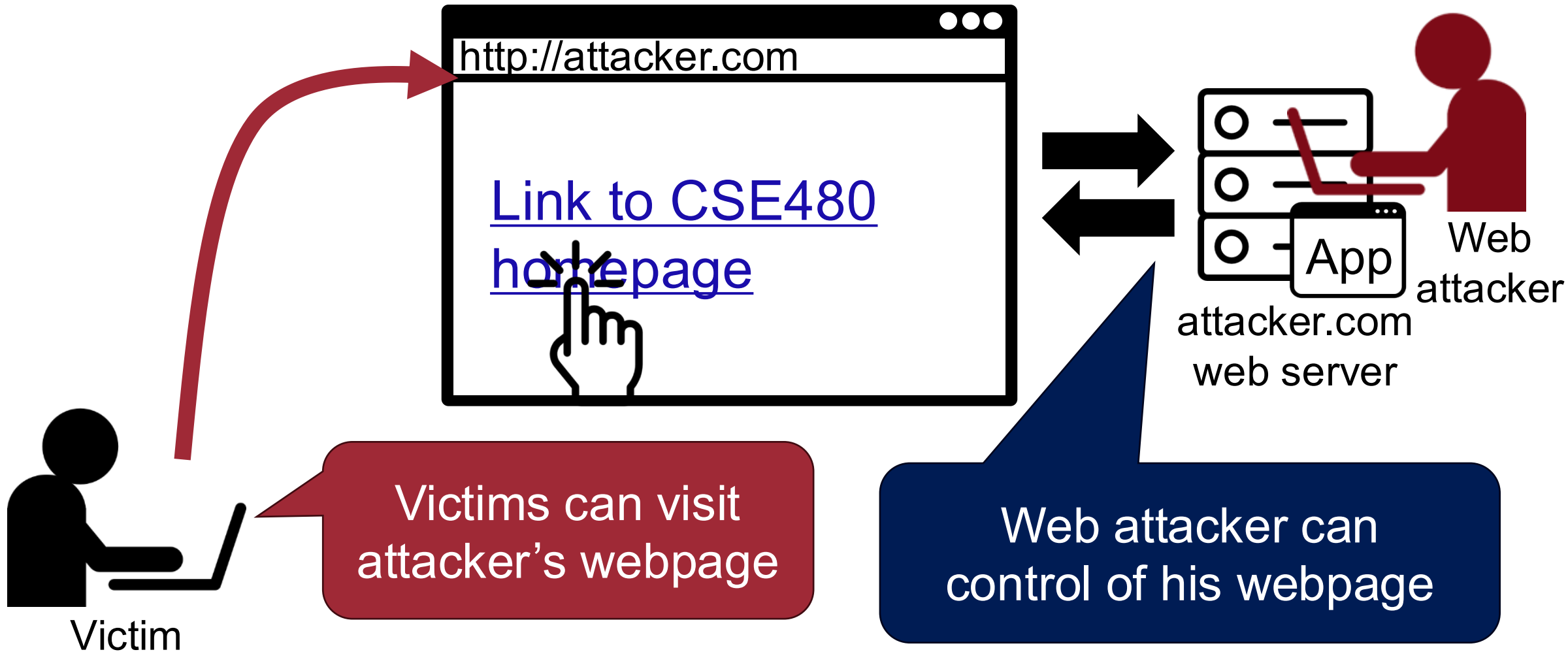


2nd try



Session Cookie Demo

Recap: Web Attacker

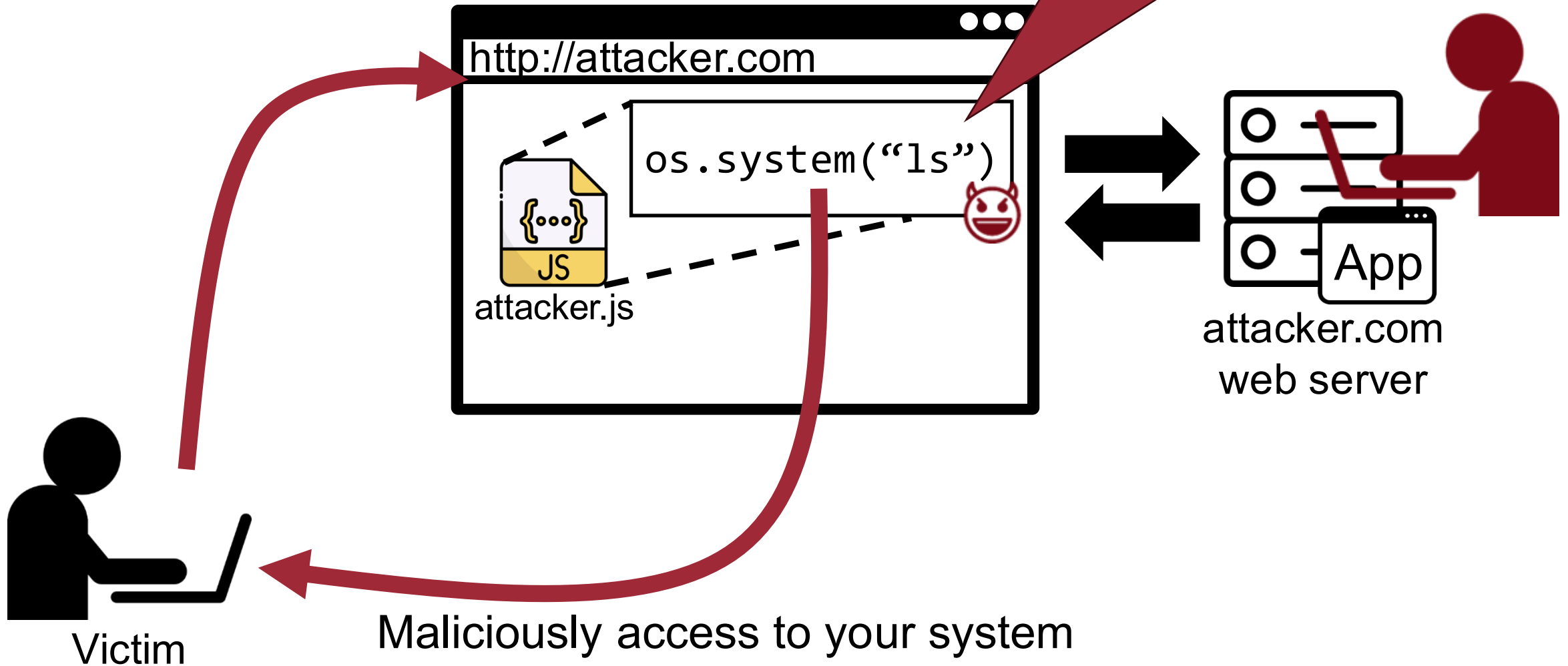


What will be happen?



Obviously some big security concerns

16

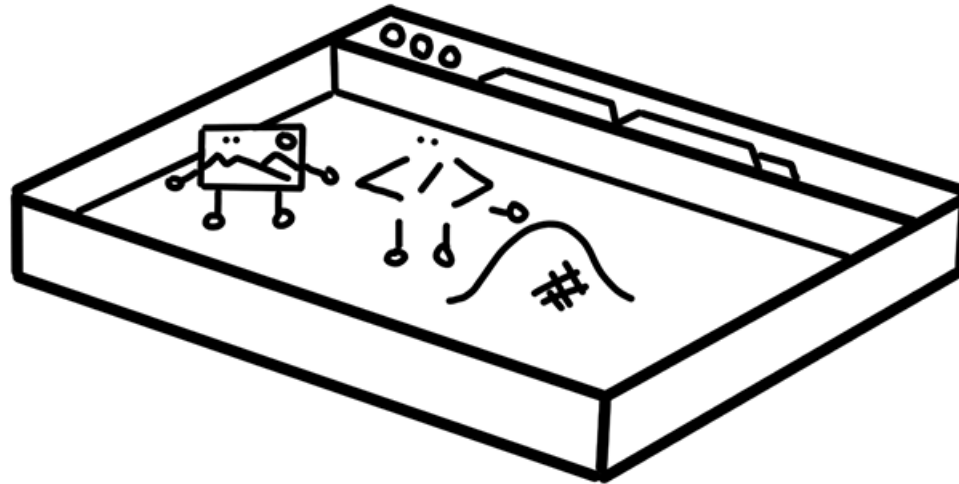


Browser Sandbox

Browser Sandbox



- No direct file access, limited access to OS
- Goal: Safely execute JavaScript code provided by a remote website
 - Isolated process when HTML rendering and JavaScript execution



Browser Sandbox Escaping Vulnerabilities¹⁹

- Related to memory-level vulnerabilities, including Use-After-Free (UAF), heap overflow,...
- CVE-2013-6632
- CVE-2014-3188
- CVE-2015-6767
- CVE-2019-5850

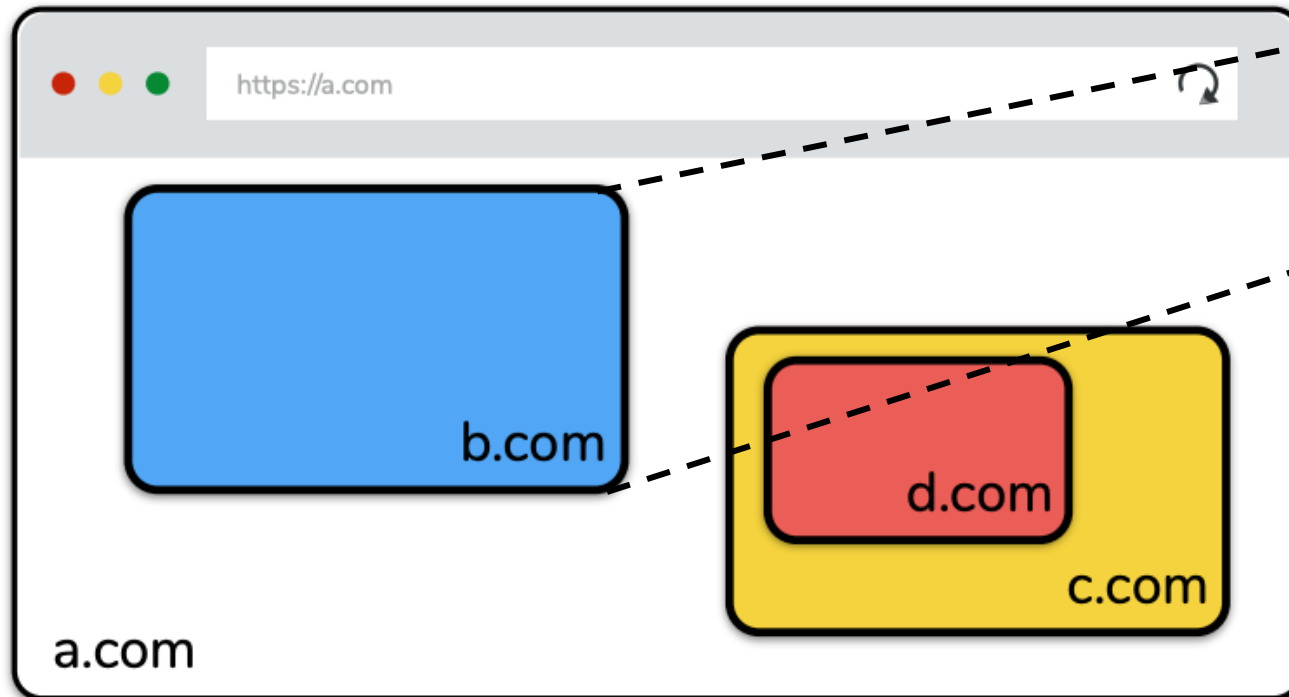
Same Origin Policy (SOP)



- One of the browser sandboxing mechanism
- The basic security model enforced in the browser

Recap: Browser Execution Model

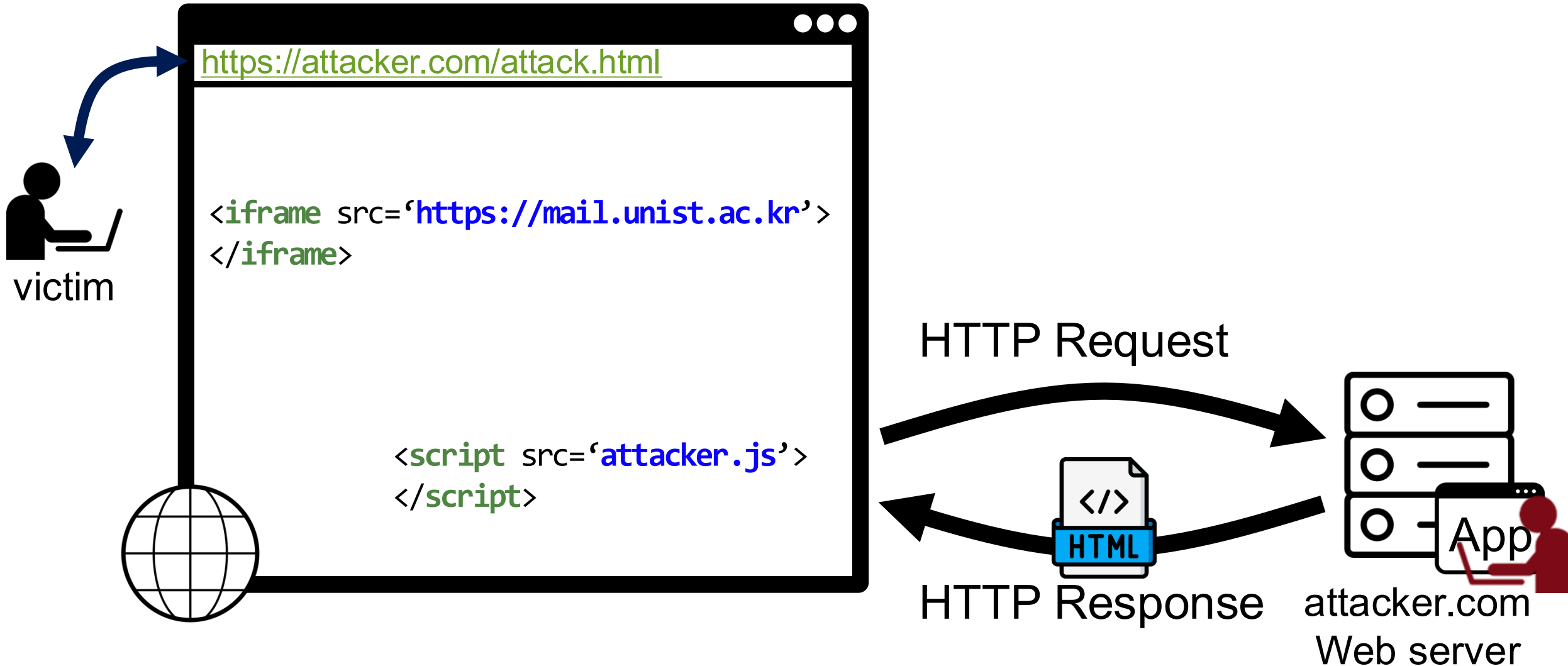
- Windows may contain frames from different sources
 - **Frame**: rigid visible division
 - **iFrame**: floating inline frame



```
<iframe src="b.com">  
</iframe>
```

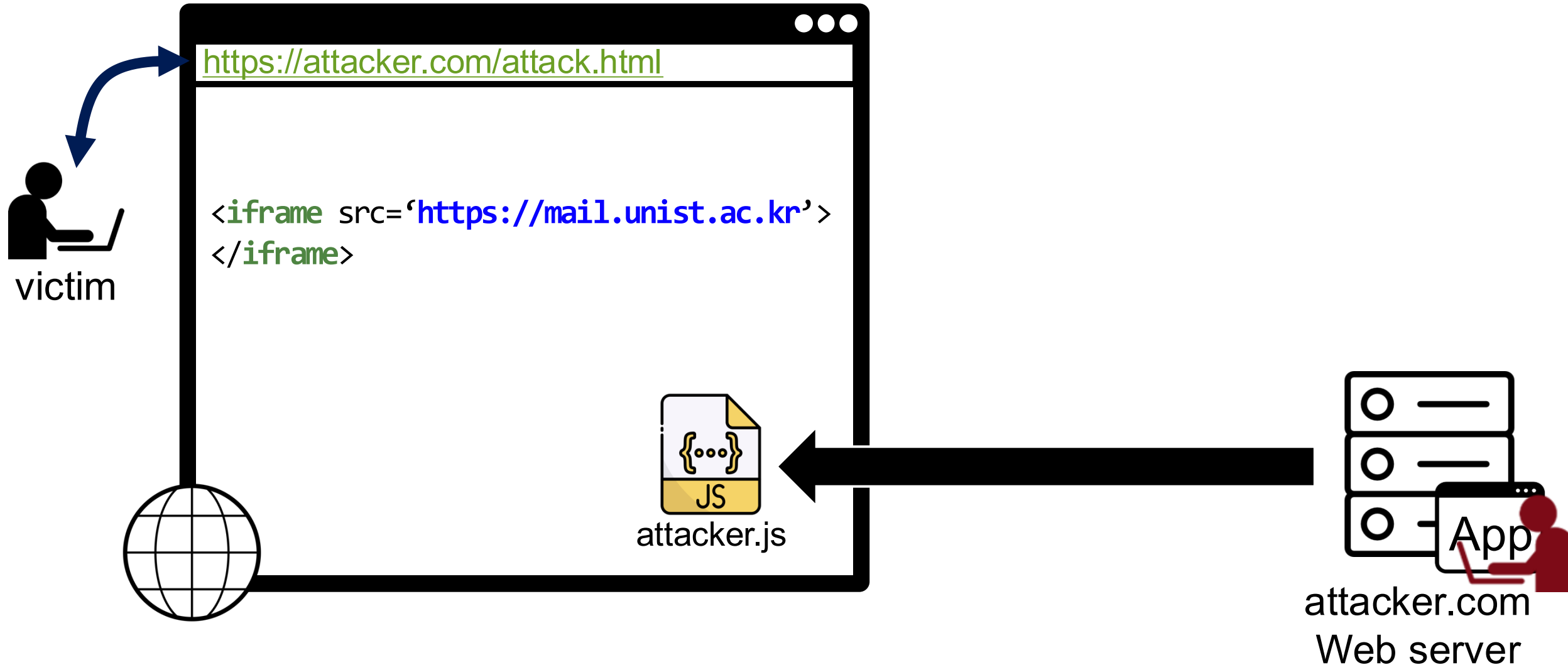
Motivative Example for SOP

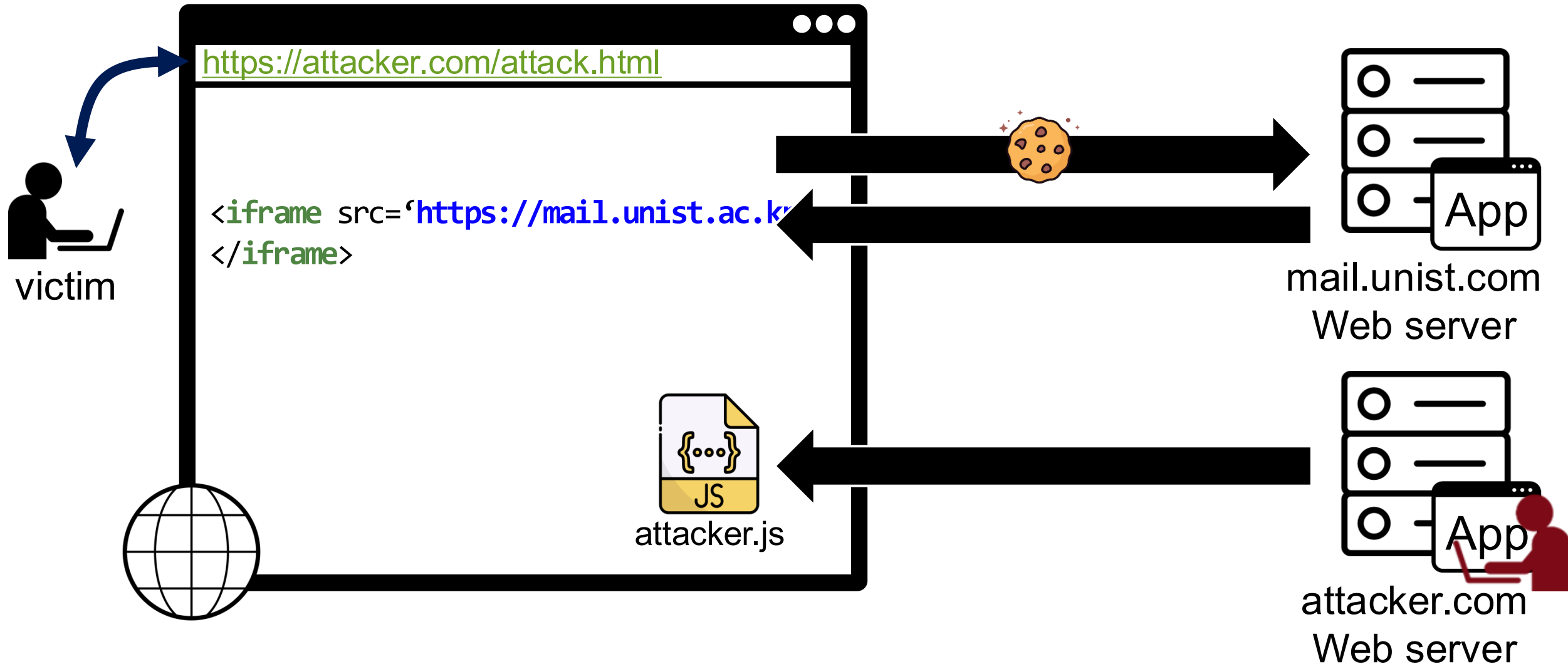
22



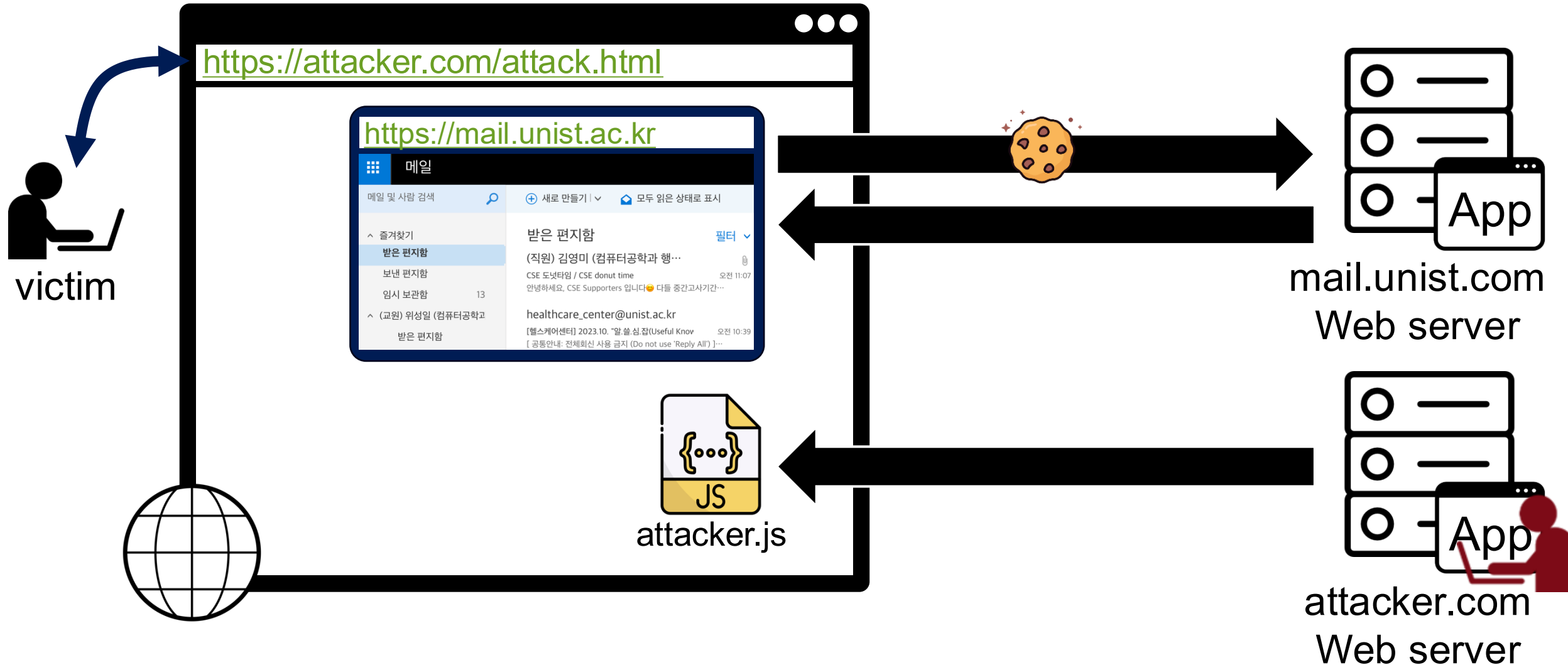
Motivative Example for SOP

23





Motivative Example for SOP



Cookie: Making HTTP Stateful



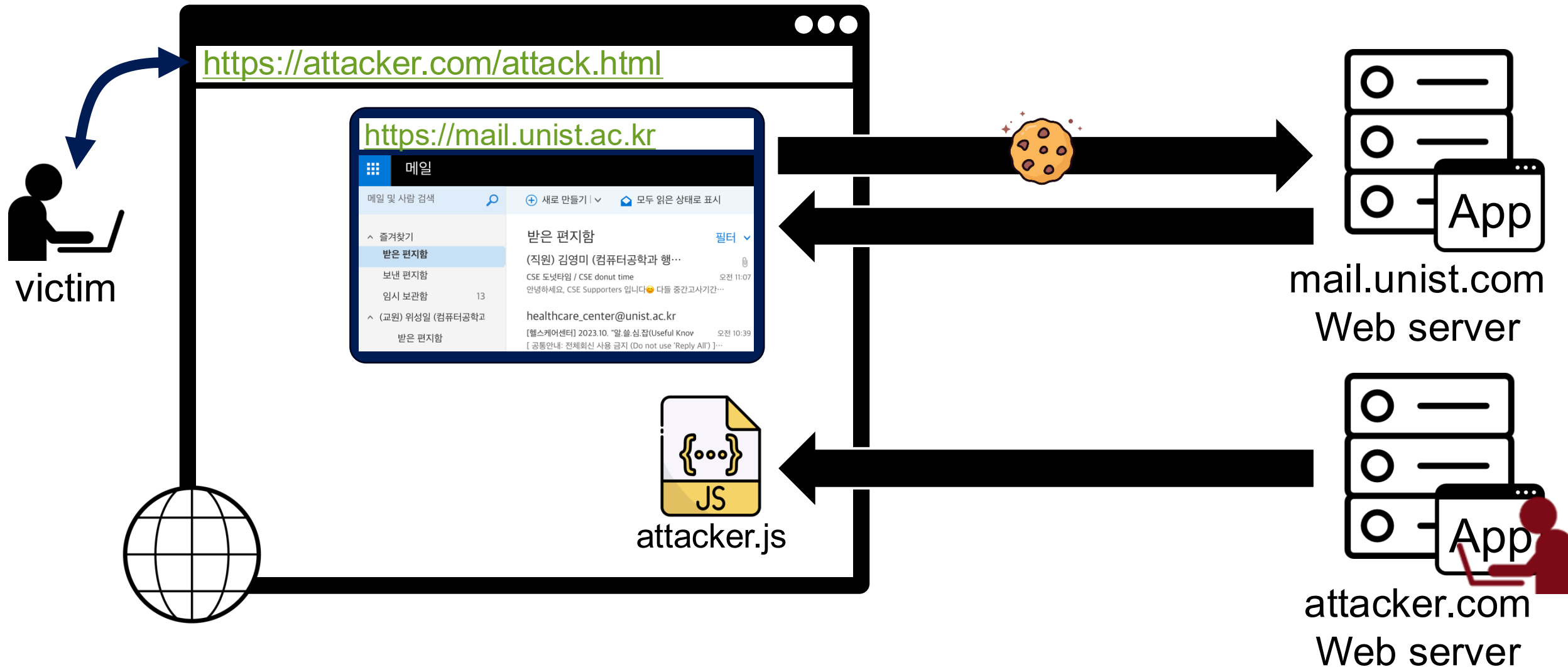
26

- Store a server-created file (cookie) in the browser
- Examples
 - Authentication (log in)
 - Personalization (language preference, shopping cart)
 - User tracking
- We can display all cookies for current document by `alert(document.cookie)`

security=low; PHPSESSID=ca5213aba0449128c7caf0902b77f1e0

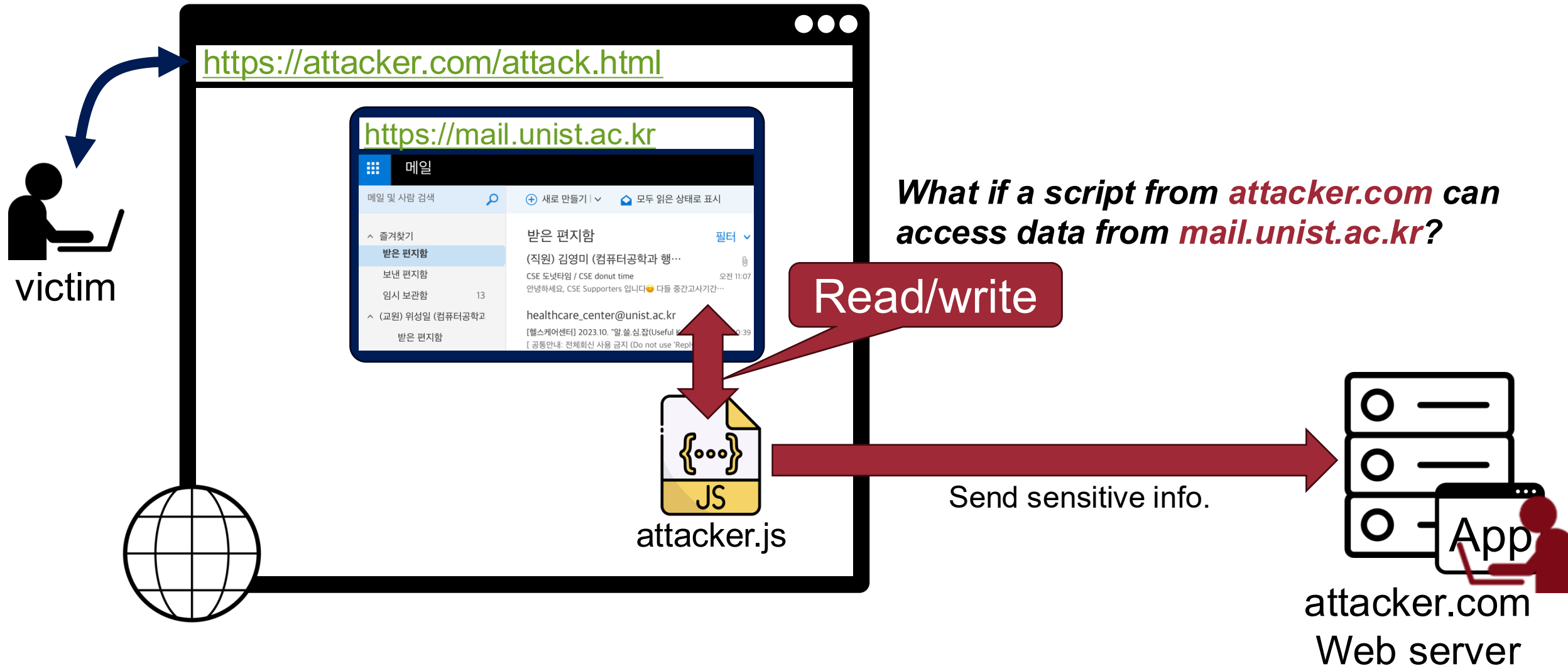
OK

Motivative Example for SOP



A World Without Separation between Sites

28





victim

<https://attacker.com/attack.html>

<https://mail.unist.ac.kr>

메일

메일 및 사람 검색

④ 새로 만들:

받은 편지함

(직원) 김영미 (컴퓨터공학과 행...

CSE 도넛타임 / CSE donut time 오전 11:07

안녕하세요, CSE Supporters 입니다😊 다들 중간고사기간...

healthcare_center@unist.ac.kr 

Healthcare_center@unlhist.ac.kr

[공통안내: 전체회식 사용 금지 (Do not use 'Repit





Page 10 of 10

1000

able to

be able to

Capitol

11

cc outb

2019

CO, DATA

What if a script from *attacker.com* can access data from *mail.unist.ac.kr*?

Read/write

It would be able to read your emails,
private messages, authentication session cookies

Motivation of the Client-side Security

30



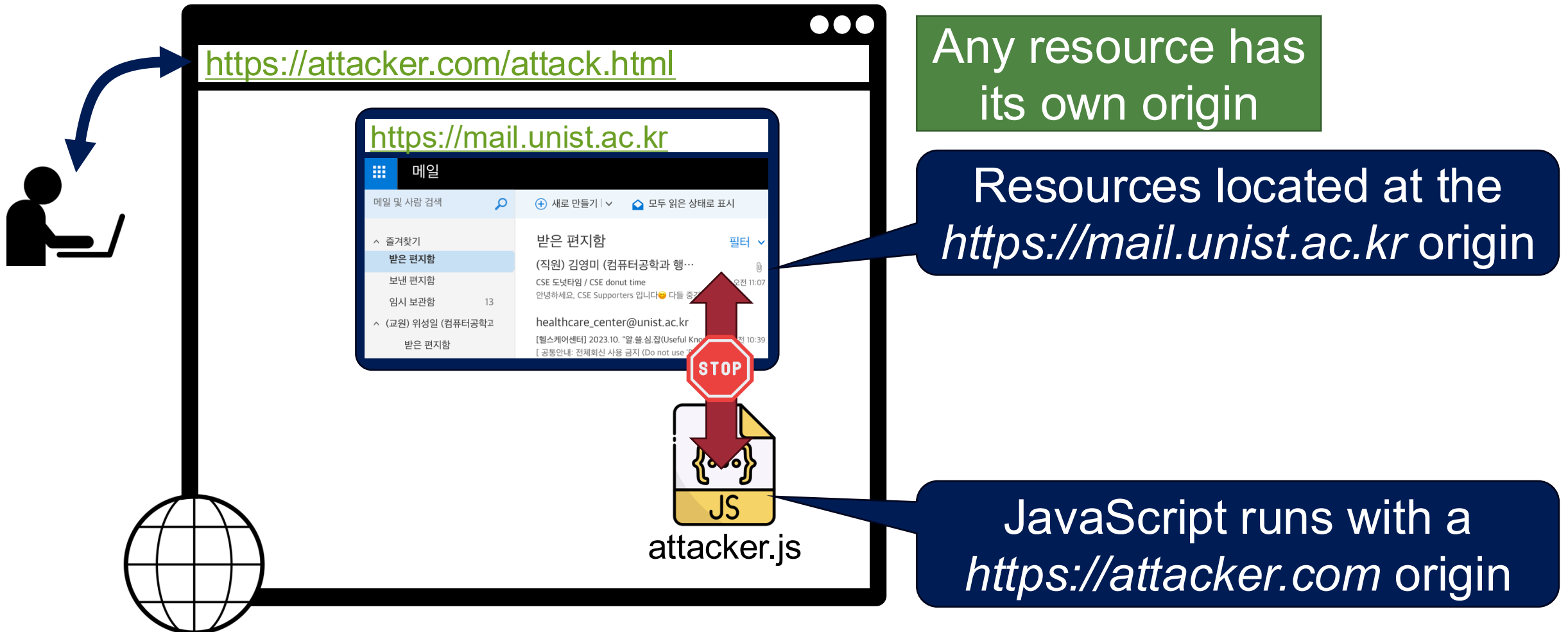
How can we prevent such malicious behaviors?

Same Origin Policy (SOP)

- Restricts scripts on **one origin** from accessing data from **another origin**

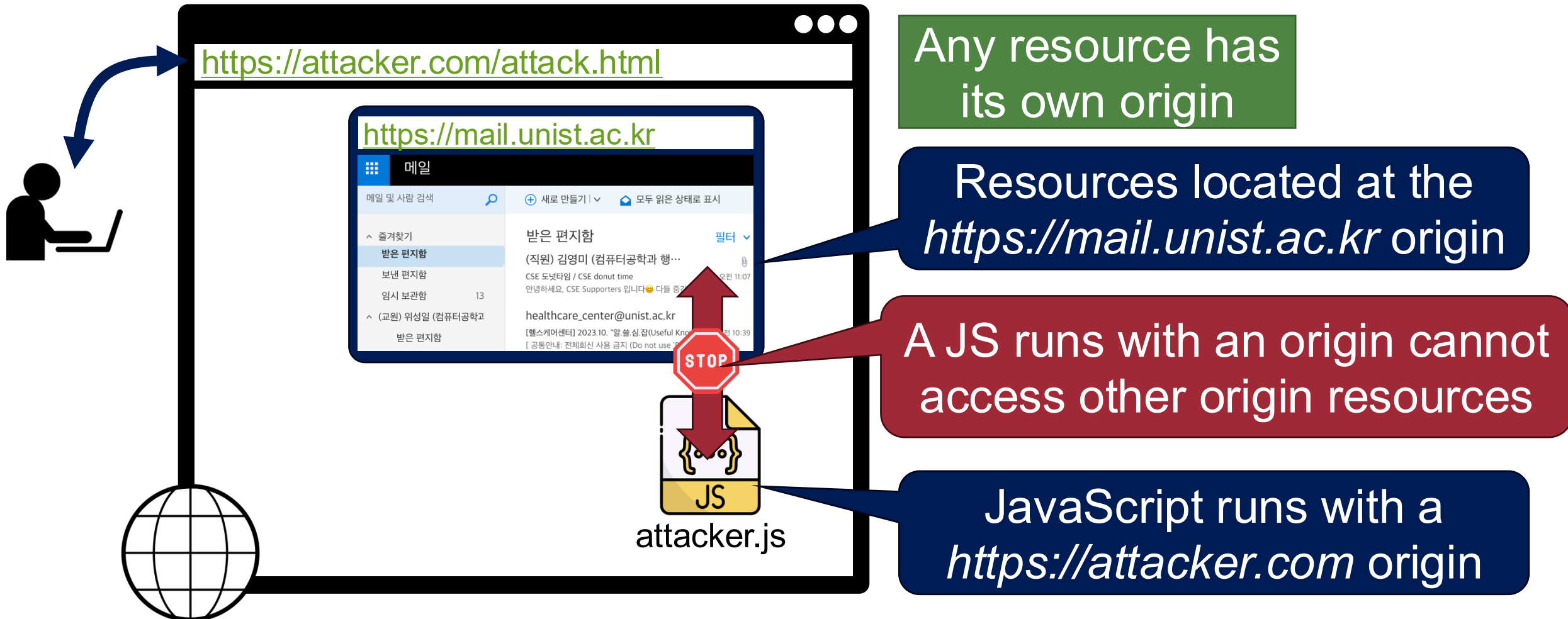
Same Origin Policy (SOP)

- Restricts scripts on **one origin** from accessing data from **another origin**



Same Origin Policy (SOP)

- Restricts scripts on **one origin** from accessing data from **another origin**



Same Origin Policy (SOP)

- Restricts scripts on **one origin** from accessing data from **another origin**



Uncaught DOMException: Permission denied to access property “document” on cross-origin object

Same Origin Policy (SOP)



- Restricts scripts on **one origin** from accessing data from **another origin**
- The basic security model enforced in the browser
- Basic access control mechanism for web browsers
 - All resources such as DOM, cookies, JavaScript has their own origin
 - SOP allows a subject to access only the objects from the same origin

What is an Origin?



- **Origin = Protocol + Domain Name + Port**
 - origin = protocol://domain:port
- Any resource has its own origin (owner)
- Two URLs have the same origin if the **protocol**, **domain name** (not subdomains), **port** are the same for both URLs
 - All three must be equal origin to be considered the same

What is an Origin?

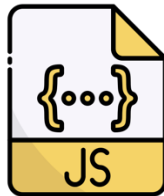


- **Origin = Protocol + Domain Name + Port**
 - origin = protocol://domain:port
- Any resource has its own origin (owner)
- Two URLs have the same origin if the **protocol**, **domain name** (not subdomains), **port** are the same for both URLs
 - All three must be equal origin to be considered the same

Demo: Same Origin Policy

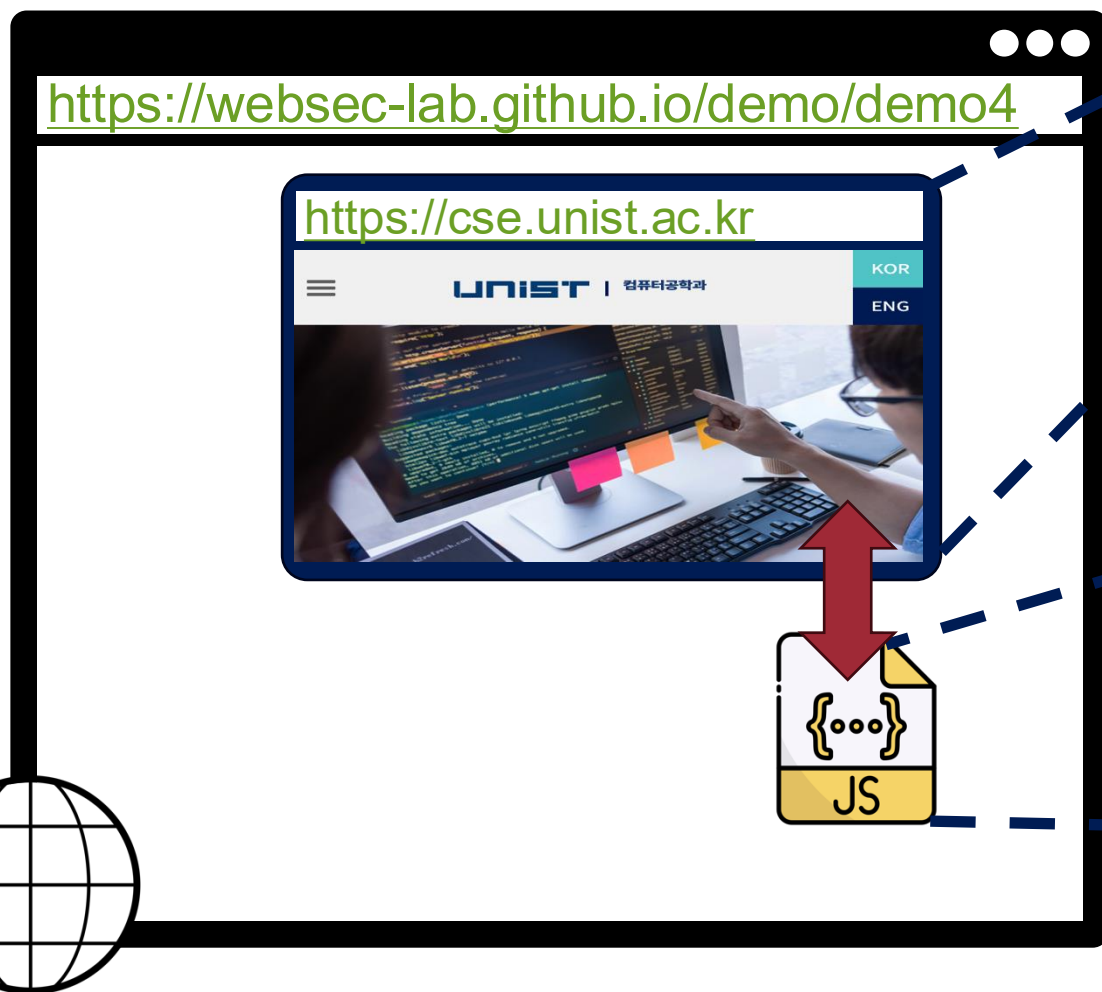
<https://websec-lab.github.io/demo/demo4>

<https://cse.unist.ac.kr>



```
<iframe id="UNIST_CSE"  
        src=https://cse.unist.ac.kr/>  
</iframe>
```

Demo: Same Origin Policy



```
<iframe id="UNIST_CSE"  
        src=https://cse.unist.ac.kr/>  
</iframe>
```

```
cookie =  
    document.getElementById('UNIST_CSE').  
        contentWindow.document.cookie;  
console.log(cookie)
```

Demo: Same Origin Policy



```
<iframe id="UNIST_CSE"  
      src=https://cse.unist.ac.kr/>  
</iframe>
```

```
cookie =  
  document.getElementById('UNIST_CSE')  
    .contentWindow.document.cookie;  
console.log(cookie);
```

Uncaught DOMException: Blocked a frame with origin "https://websec-lab.github.io" from accessing a cross-origin frame

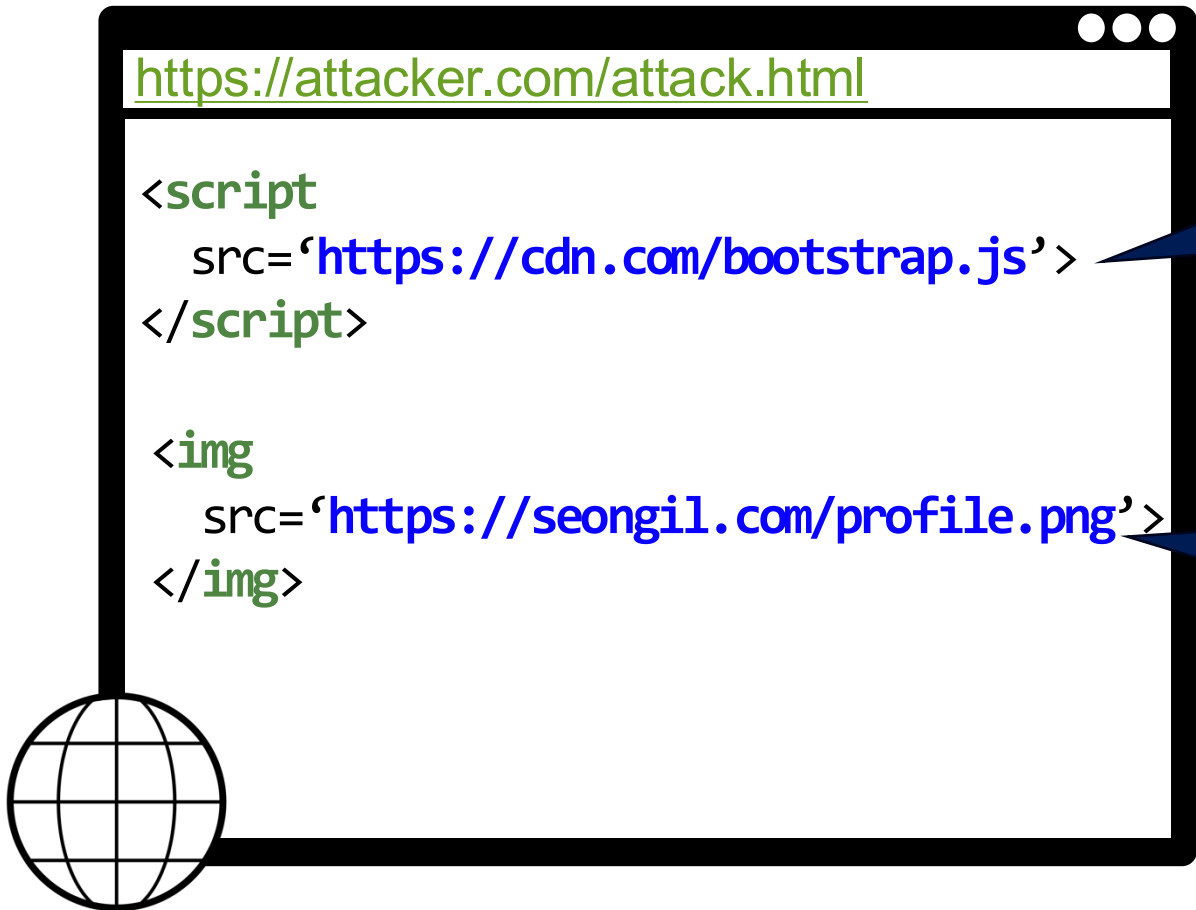
DEMO

<https://websec-lab.github.io/courses/2026f-cse480/demo/demo4.html>

For Your Information...



- **Cross-origin loading** of page resources is generally permitted
 - E.g., the SOP allows embedding of external resources via HTML tags (e.g., `<img>`, `<video>`, `<script>`, ...)



The origin of the loaded script is `https://attacker.com`

The origin of the loaded image is `https://attacker.com`

Analogy

- **Operating system**

- Primitives (Resources)
 - System calls
 - Processes
 - Disk
- Principals: Users
 - Discretionary access control
- Vulnerabilities
 - Buffer overflow
 - Root exploit

- **Web browser**

- Primitives (Resources)
 - Document object model
 - Frames
 - Cookies / localStorage
- Principals: “Origins”
 - Mandatory access control
- Vulnerabilities
 - Cross-site scripting
 - Cross-site request forgery
 - Cache history attacks
 - ...

Cookie Security

How to make HTTP stateful securely?

Same Origin Policy: “High Level”

- Recap: Same Origin Policy (SOP) for DOM:

Origin A can access origin B's DOM if match on:
(protocol, domain, port)

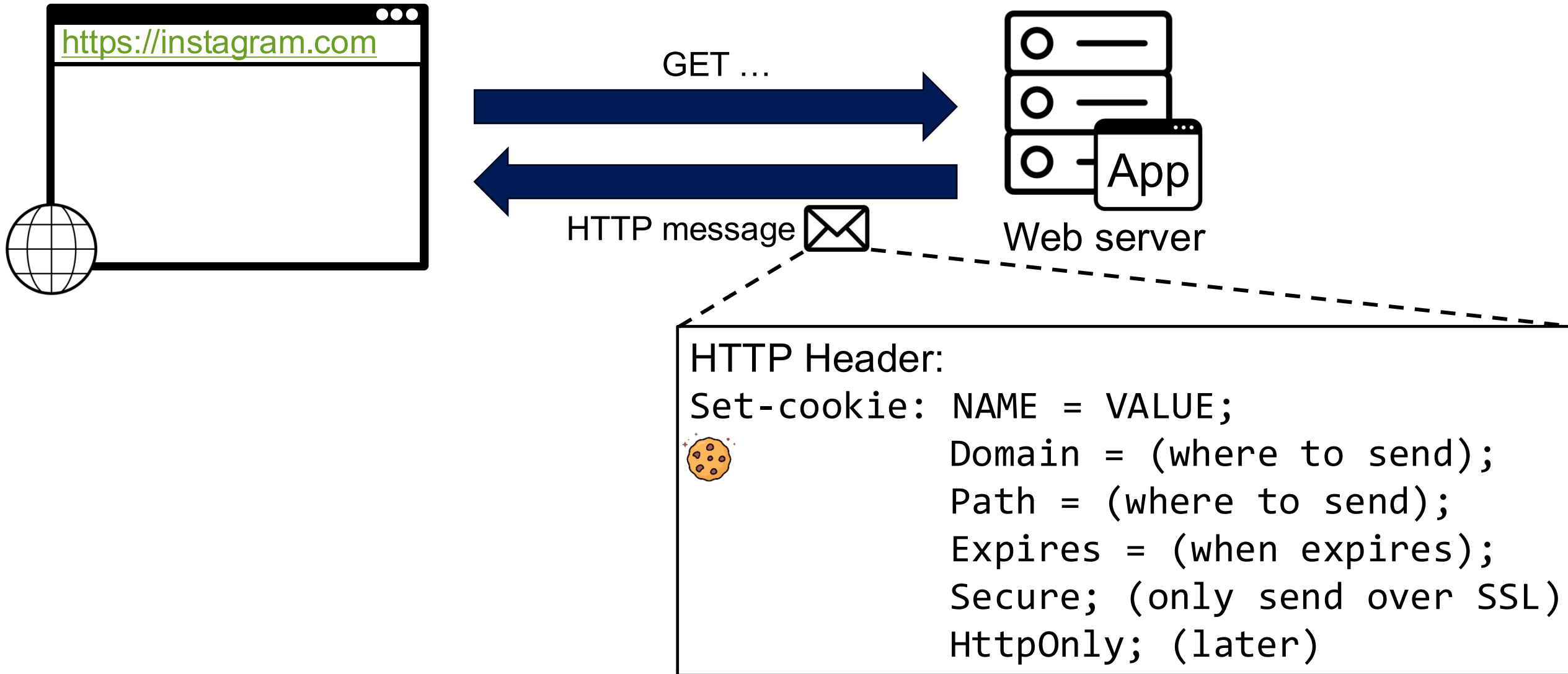
- Today: Same Origin Policy (SOP) for cookies: 

Generally speaking, based on:
([protocol], domain, *path*)

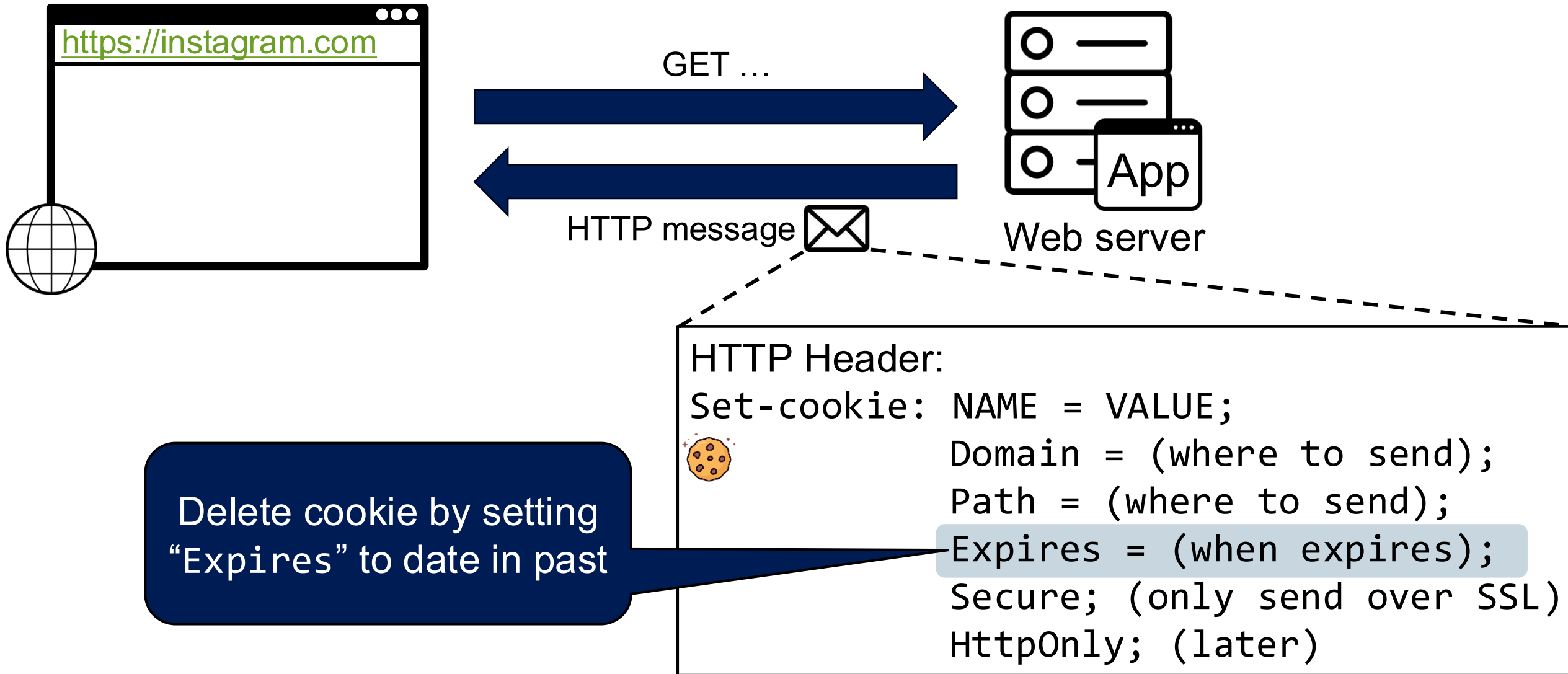
Optional

`protocol://domain:port/path?params`

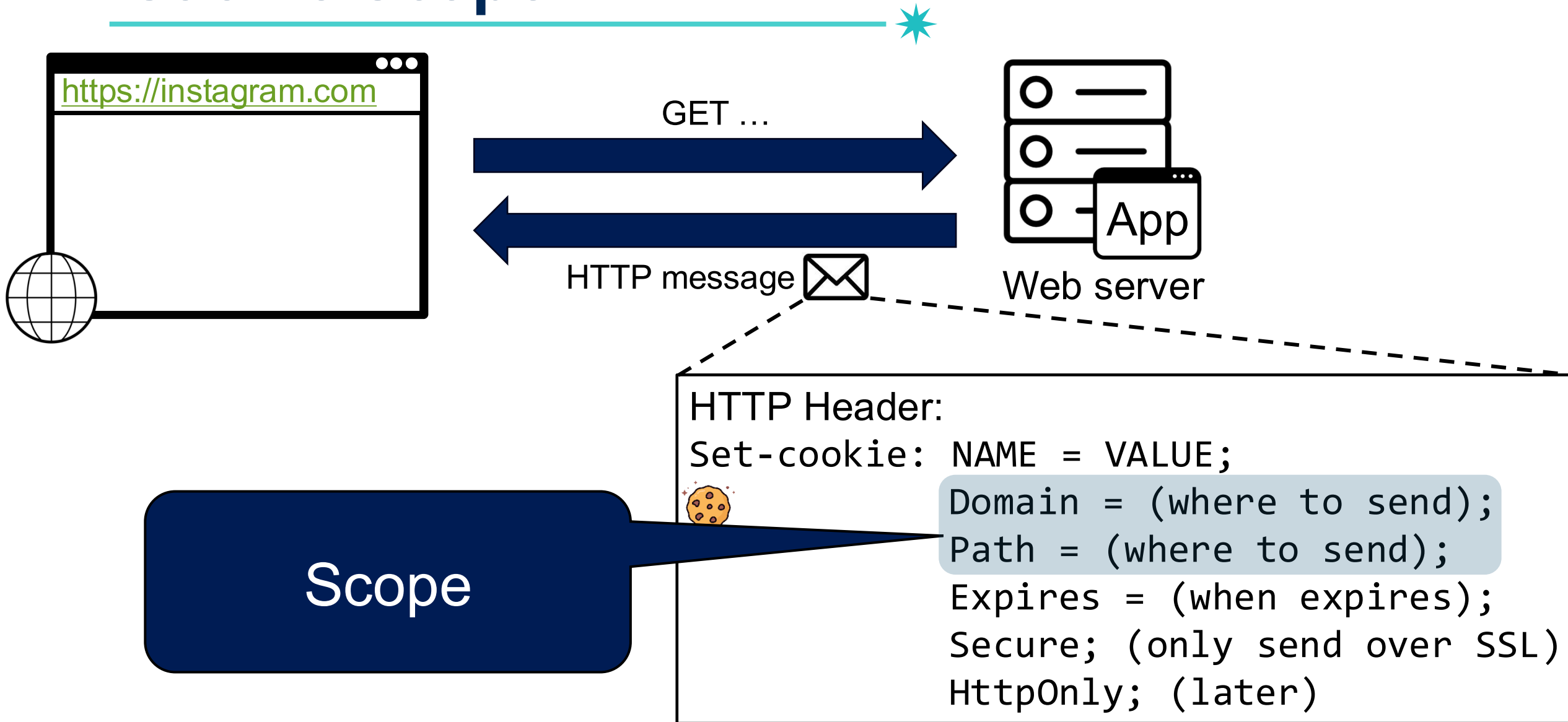
Setting Cookies by Server



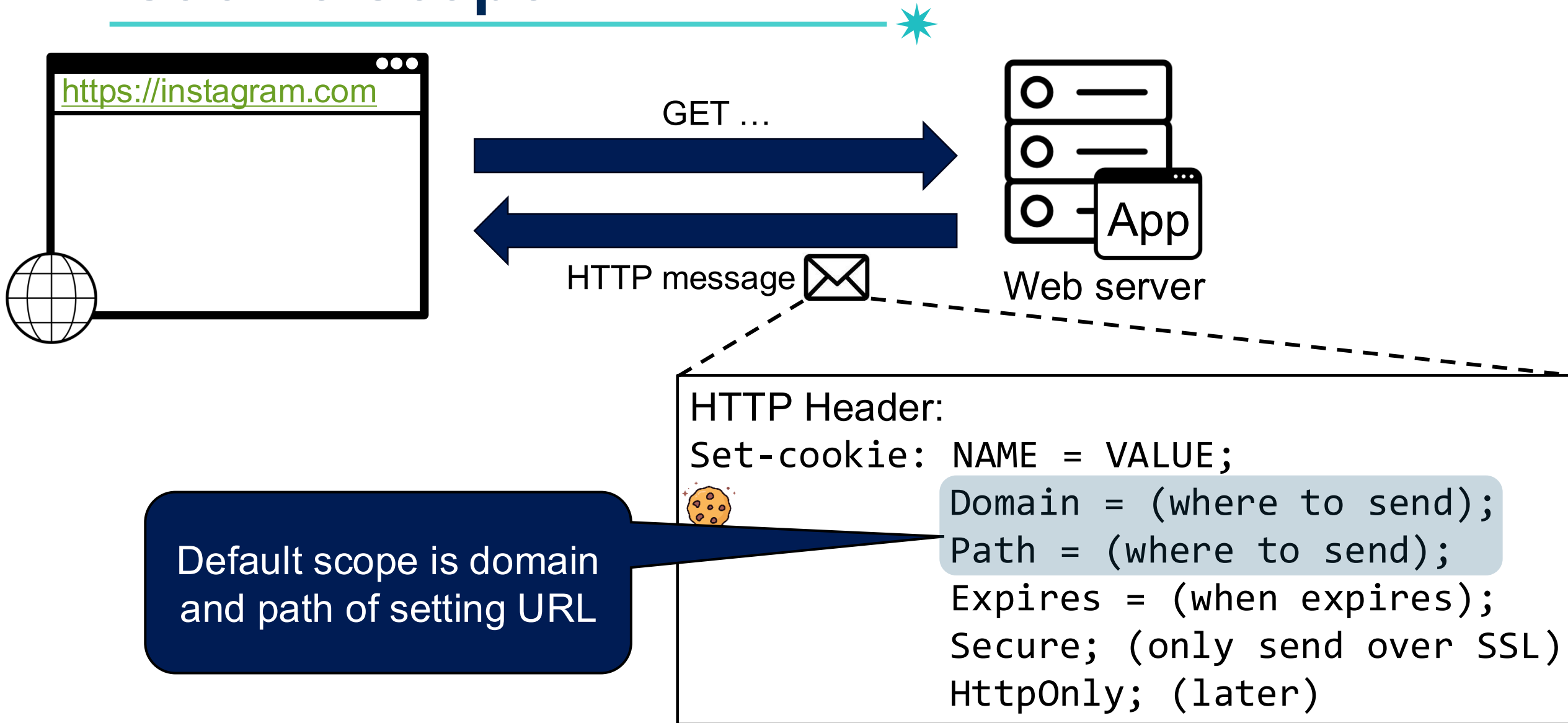
Deleting Cookies by Server



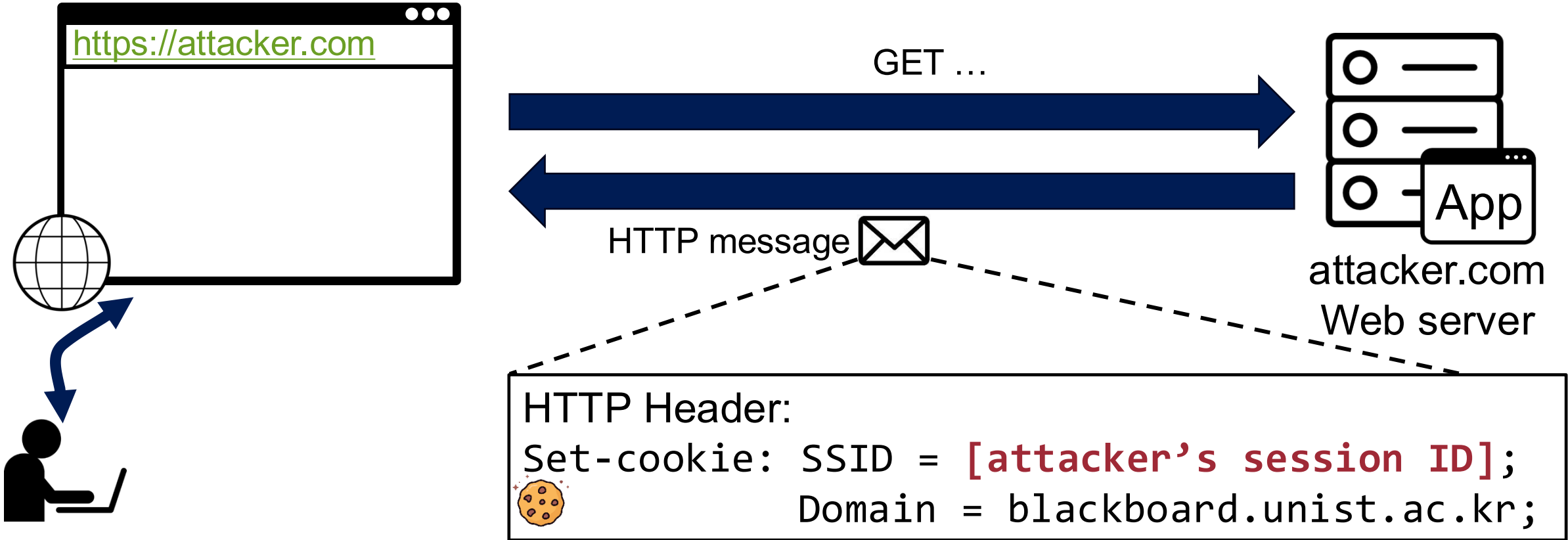
Cookie Scope



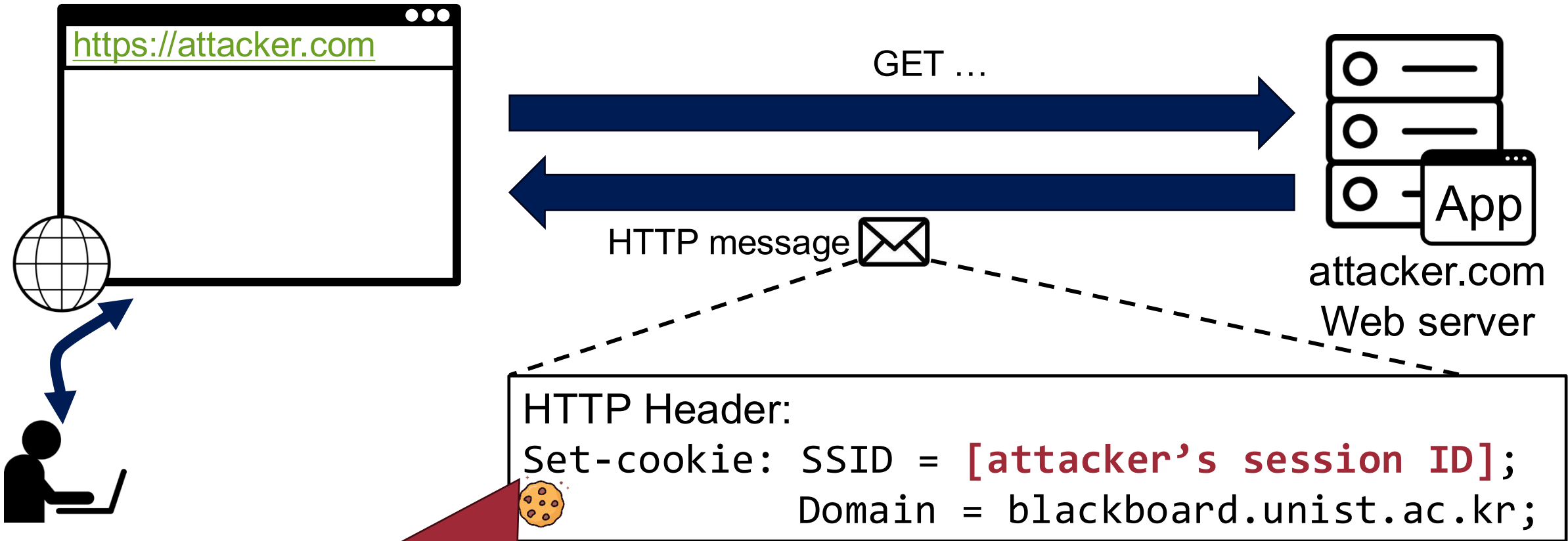
Cookie Scope



Motivating Example of the Cookie Write SOP



Motivating Example of the Cookie Write SOP



The attacker can write arbitrary values to the arbitrary cookies

Scope Setting Rules (Write SOP)



- **Domain:** any domain-suffix of URL-hostname, except Top Level Domain (TLD)
- **Path:** can be set to anything

Scope Setting Rules (Write SOP)

- **Domain:** any domain-suffix of URL-hostname, except Top Level Domain (TLD)

Question: which cookies can be set by login.site.com?

Idx	Cookie's domain	Write Allowed?
1	login.site.com	
2	.site.com	
3	.com	
4	alice.site.com	
5	othersite.com	

- **Path:** can be set to anything

Cookies are Identified by (name, domain, path)

Cookie 1 🍪

name = **userid**

Value = test

domain = **login.site.com**

Path = /

secure



Cookie 2 🍪

name = **userid**

Value = test123

domain = **.site.com**

Path = /

secure

Distinct
cookies

Browser's Cookie Jar



Cookie 1 🍪

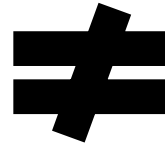
name = **userid**

Value = test

domain = **login.site.com**

Path = /

secure



Cookie 2 🍪

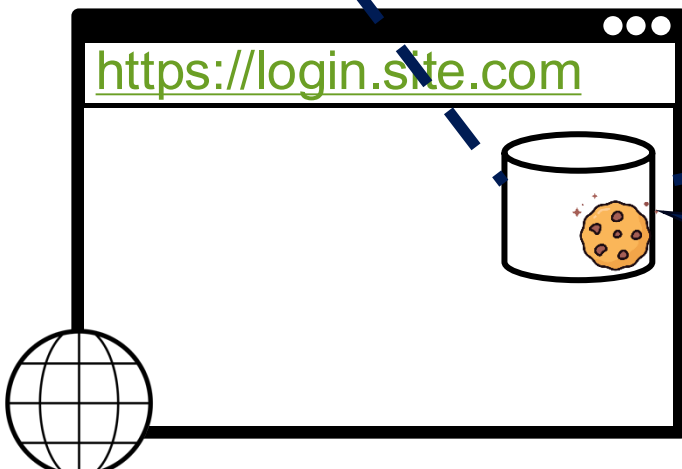
name = **userid**

Value = test123

domain = **.site.com**

Path = /

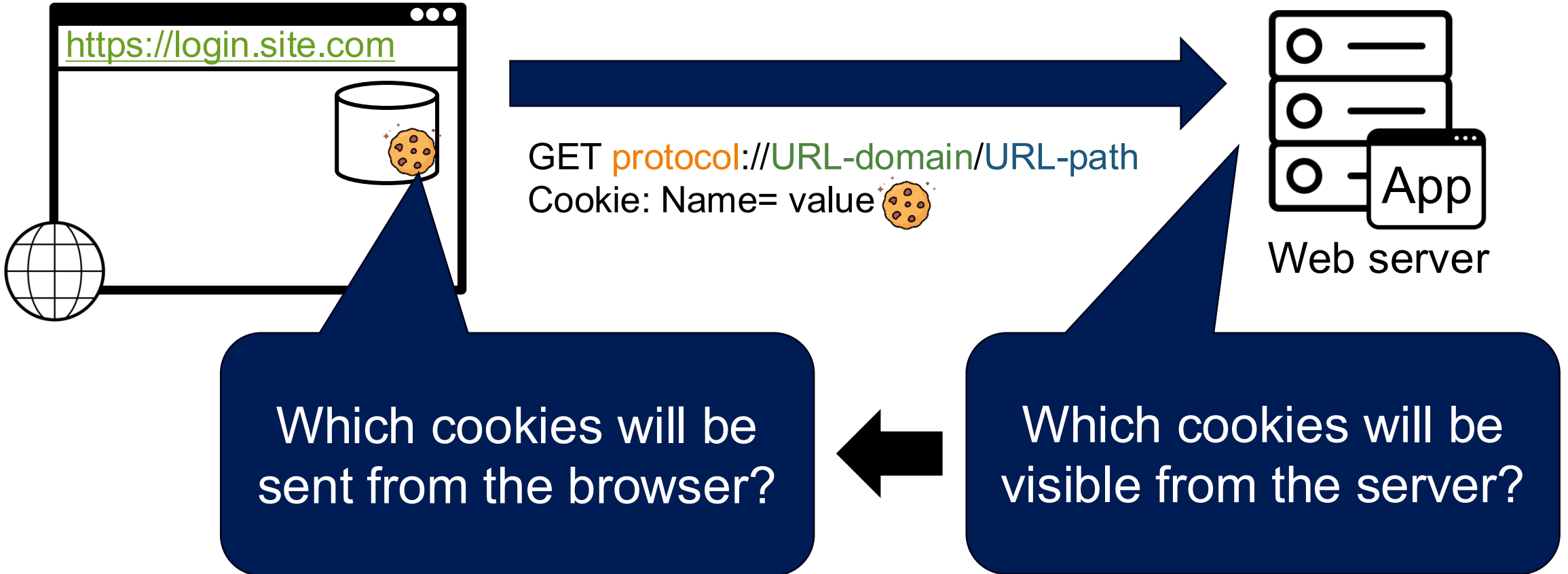
secure



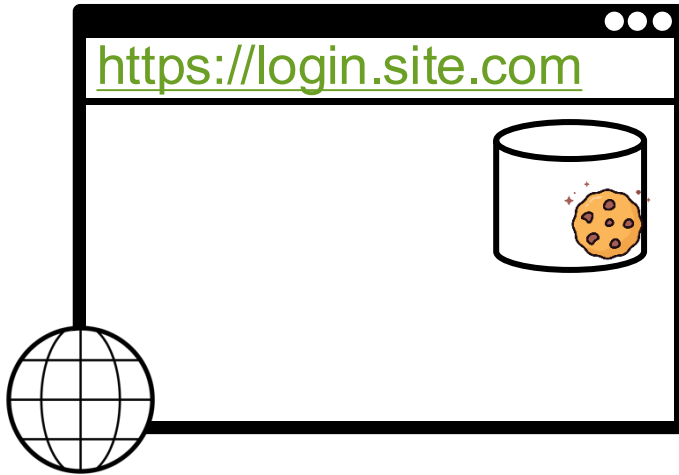
Both cookies are stored in browser's cookie jar;
Both are in scope of login.site.com

Reading Cookies on Server (Read SOP)

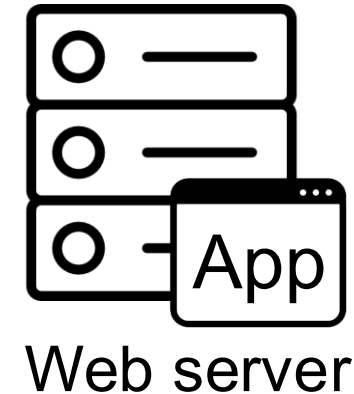
59



Reading Cookies on Server (Read SOP)



GET `protocol://URL-domain/URL-path`
Cookie: Name= value 



- **Browser sends all cookies in URL scope:**
 - Cookie domain is domain-suffix of `URL-domain`, and
 - Cookie path is prefix of `URL-path`, and
 - [`protocol`=HTTPS if cookie is “secure”]
- **Goal:** server only sees cookies in its scope

Client-side Read/Write: `document.cookie`

63



- Setting a cookie in JavaScript:
 - E.g., `document.cookie = "name=value; expires=...;"`
 - E.g., `document.cookie = "ssid=AB31FBS5; domains=.unist.ac.kr"`
- Reading a cookie: `alert(document.cookie)`
 - Prints string containing all cookies available for document
- Deleting a cookie:
 - `Document.cookie = "name=; expires=Thu, 01-Jan-70"`

Client-side Read/Write: `document.cookie`

64

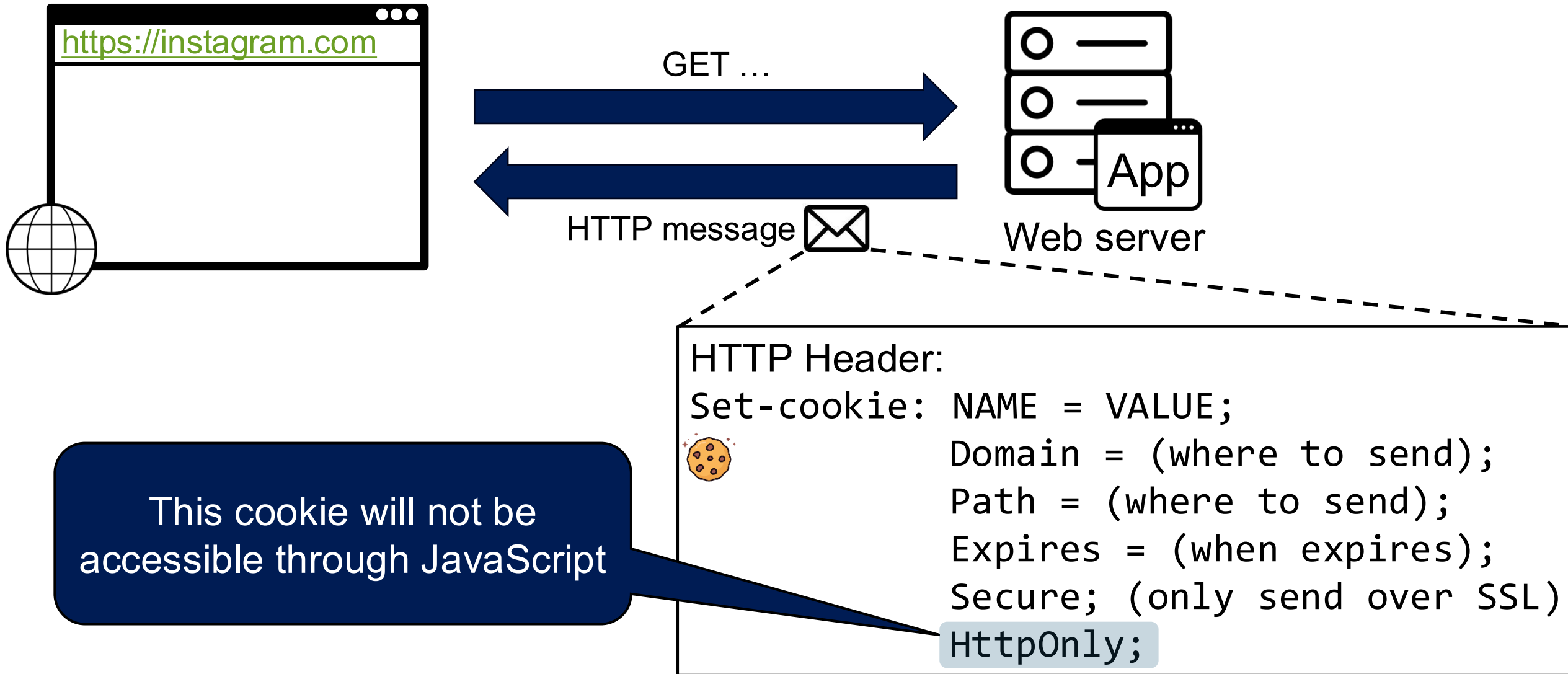
We can access all cookies for current document by
`alert(document.cookie)`

An alert dialog box with a light gray background and a thin black border. It contains a single line of text in a monospaced font: 'security=low; PHPSESSID=ca5213aba0449128c7caf0902b77f1e0'. In the bottom right corner, there is an 'OK' button with a dashed border.

`security=low; PHPSESSID=ca5213aba0449128c7caf0902b77f1e0`

OK

HttpOnly Cookies



HttpOnly Cookies



- Cookie sent over HTTP(s), but not accessible to scripts
 - Cannot be read via `document.cookie`
 - Helps prevent cookie theft attacks

HttpOnly Cookie Demo

Question?